Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
—
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 5

«Системные вызовы»

по дисциплине «Операционные системы»

Выполнил
студент гр. 5151003/40001 Корпусов Д.Н

<подпись>

Руководитель Югай П.Э

<подпись>

Санкт-Петербург
2025

Содержание

Цель работы 3
Ход работы 3-36
1. Теоретическая часть 3-4
2. Практическая часть 4-36
Индивидуальное дополнительное задание 37-38
Вывод 39
Листинг А «syscall.c» 40-45
Листинг Б «editor.c» 46-54

ЦЕЛЬ РАБОТЫ

Цель лабораторной работы – изучение механизма внутренних прерываний и системных вызовов, модификация обработчика системных вызовов и внедрение набора системных вызовов в учебную операционную систему Pintos.

ХОД РАБОТЫ

1. Теоретическая часть

Системные вызовы представляют собой основной механизм взаимодействия пользовательских программ с ядром операционной системы. В архитектуре Pintos они выступают в роли «моста», через который процессы пользовательского режима получают доступ к возможностям ядра: работе с файлами, управлению процессами, завершению программы, вводу-выводу и т.д. В пользовательском пространстве системный вызов представляет собой обычную функцию, но её выполнение приводит к возникновению внутреннего прерывания 0x30, которое переключает процессор в привилегированный режим. Именно это даёт ядру возможность безопасно управлять ресурсами системы и защищать память от некорректного доступа.

Когда пользовательская программа вызывает системный вызов, процессор автоматически сохраняет состояние всех регистров в структуру intr_frame. Эта структура передаётся ядру как снимок состояния процесса в момент вызова. Обработчик прерываний, ассоциированный с вектором 0x30, получает указатель на эту структуру и может использовать её поля как входные данные. Кроме того, изменение определённых регистров (например, eax) в intr_frame позволяет вернуть значение пользователю. После завершения работы обработчика прерываний система восстанавливает контекст процесса и продолжает его выполнение так, будто это обычный вызов функции.

По соглашению Pintos номер системного вызова и его аргументы располагаются в пользовательском стеке, начиная с адреса esp. Первый элемент — номер вызова, последующие — аргументы. Обработчик

системных вызовов должен аккуратно интерпретировать эти данные, предварительно проверив, что указанные адреса принадлежат пользовательскому адресному пространству. Проверка корректности выполняется функциями is_user_vaddr() и pagedir_get_page(). Любая попытка использовать недействительный указатель рассматривается как ошибка пользователя и приводит к аварийному завершению процесса через вызов exit(-1).

Одним из ключевых принципов реализации системных вызовов является обеспечение безопасности ядра. Пользовательская программа может намеренно или случайно передать недопустимый указатель, который указывает за пределы выделенной памяти или внутрь пространства ядра. Если обработчик попытается разыменовать такой указатель, это может привести к нарушению работы всей системы. Поэтому в Pintos перед чтением любого аргумента выполняется проверка корректности адреса. В случае нарушения правила обработчик немедленно завершает программу пользователя, предотвращая утечку ресурсов и повреждение данных.

Для выполнения лабораторной работы необходимо модифицировать обработчик syscall_handler таким образом, чтобы он корректно считывал номер системного вызова, интерпретировал его аргументы и возвращал результат через регистр eax структуры intr_frame. Улучшенный обработчик выполняет последовательность шагов: проверка указателя esp, получение номера вызова, чтение аргументов, вызов соответствующей внутренней функции ядра и запись результата обратно в f->eax. Такая схема создаёт универсальный механизм, позволяющий расширять набор поддерживаемых системных вызовов без изменения архитектуры ядра.

2. Практическая часть

В ходе выполнения лабораторной работы был модифицирован обработчик внутренних прерываний, отвечающий за системные вызовы пользовательских программ. Базовая версия Pintos не реализует полноценный механизм передачи аргументов, поэтому в функцию syscall_handler() были внесены изменения, позволяющие считать номер системного вызова и его аргументы непосредственно из пользовательского стека. Адрес начала стека передаётся ядру через поле esp структуры intr_frame, в которой сохраняется контекст выполнения процесса на момент прерывания. (см. рис. 1)

```
static void
syscall_handler (struct intr_frame *f)
{
  uint32_t *usp = (uint32_t *) f->esp;

  validate_user_pointer(usp);
  int syscall_number = usp[0];
```

Рис. 1

Первым шагом была добавлена проверка корректности указателя esp. Поскольку пользователь может передать недопустимый адрес, ядро обязано убедиться, что данный указатель указывает на действительную страницу пользовательской памяти. В проекте реализована функция validate_user_pointer() (см. рис. 2), проверяющая принадлежность адреса пользовательскому адресному пространству и наличие физической страницы, отображённой по этому виртуальному адресу. При нарушении условия процесс аварийно завершается вызовом exit(-1), что предотвращает повреждение памяти ядра.

```
static void validate_user_pointer(const void *uaddr)
{
  if (uaddr == NULL || !is_user_vaddr(uaddr)
      || pagedir_get_page (thread_current()->pagedir, uaddr)==NULL)
  {
    sys_exit(-1);
  }
}
```

Рис. 2

После проверки esp обработчик интерпретирует стек как массив 32-битных значений. Первый элемент массива является номером системного вызова, а начиная со второго располагаются аргументы. Считанные данные используются для вызова соответствующей реализации системного вызова. Возвращаемое значение ядро помещает в регистр eax структуры intr_frame, откуда пользовательская программа затем получает результат.

Название теста	**Описание теста**	**Решение**
halt	Тест проверяет корректность работы системного вызова halt(), который должен немедленно завершить работу Pintos. При запуске тест вызывает функцию halt из пользовательской программы и ожидает, что ядро корректно выключит виртуальную машину. Если система не завершает работу или происходит аварийное завершение, тест считается проваленным.	Для прохождения теста в обработчик системных вызовов был добавлен новый case для номера системного вызова SYS_HALT. В рамках его обработки выполняется прямой вызов функции shutdown_power_off(), реализованной в модуле devices/shutdown.c. Эта функция корректно завершает работу Pintos, закрывая файловую систему и инициируя выключение виртуальной машины. Таким образом, системный вызов halt() работает правильно и тест завершается

		успешно.
exit	Тест проверяет, что системный вызов exit(status) немедленно завершает текущий процесс. Пользовательская программа вызывает exit(57), и, если функция работает корректно, дальнейший код (вызов fail()) не должен выполниться. Таким образом, тест убеждается, что процесс завершается сразу же после вызова exit и не продолжает выполнение инструкций.	Для прохождения теста в обработчике системных вызовов добавлена обработка номера SYS_EXIT. Аргумент status считывается из пользовательского стека, затем вызывается sys_exit(status). Реализация присваивает код завершения в thread_current()->exit_status и вызывает thread_exit(), который немедленно завершает поток, не позволяя исполнению продолжиться. Благодаря этому строка fail("should have called exit(57)") не выполняется, и тест успешно проходит.
exec-once	Тест проверяет корректность	Реализация системного

	работы системных вызовов exec() и wait() при запуске одного дочернего процесса. Пользовательская программа вызывает exec("child-simple"), затем сразу вызывает wait() для полученного PID. Ожидается, что дочерний процесс успешно загрузится, выполнится и завершится с кодом 0, а родительский процесс корректно дождётся его завершения через wait(). Любое нарушение синхронизации (например, если exec() вернёт -1 или wait() не получит корректный статус) приводит к провалу теста.	вызова exec() потребовала внесения нескольких изменений в механизм создания процессов и обработки системных вызовов. Во-первых, была добавлена структура синхронизации между родительским и дочерним процессами, основанная на семафоре sema_load. В process_execute() создаётся структура child_status, в которой хранится флаг успешной загрузки load_success. После запуска нового потока через thread_create() родительский процесс вызывает sema_down(&status->sema_load) и блокируется до тех пор, пока дочерний процесс не завершит начальный этап загрузки программы. Такой
exec-multiple	Тест проверяет корректную работу системных вызовов exec() и wait() при последовательном запуске нескольких дочерних процессов. Если хотя бы один из процессов завершается ошибкой загрузки, если exec() неверно возвращает -1, если wait() получает неправильный PID или повторное ожидание приводит к сбою, тест	

	считается проваленным.	подход устраняет гонки между exec() и start_process() и гарантирует, что exec() возвращает корректный PID только после того, как программа действительно удалось загрузить. Во-вторых, в start_process() реализована передача результата загрузки родителю. После вызова функции load() устанавливается значение status->load_success, а затем вызывается sema_up(&status->sema_load), что разблокирует process_execute(). В случае ошибки загрузки («файл не найден», неверный ELF, недостаток памяти) родитель получает отрицательный результат и системный
exec-arg	Тест проверяет корректность передачи аргументов из системного вызова exec() в дочерний процесс. Программа вызывает exec(«child-args childarg»), после чего ожидает завершения дочернего процесса через wait(). Запущенная программа child-args анализирует содержимое пользовательского стека и проверяет правильность структуры аргументов: имя программы должно быть передано как argv[0], строка childarg — как argv[1], количество аргументов должно быть корректным, а указатели в массиве argv[] должны ссылаться на верные адреса внутри пользовательского стека.	
exec-missing	Тест проверяет реакцию системного вызова exec() на попытку запуска несуществующего исполняемого файла. Пользовательская программа вызывает exec("no-such-file"),	

	где указанного файла заведомо нет в файловой системе. Согласно требованиям, в случае ошибки загрузки исполняемого файла системный вызов exec() обязан вернуть значение -1.	вызов exec() возвращает -1, что требуется в тесте exec-missing. Если же загрузка прошла успешно, родитель получает корректный PID, что необходимо для успешного выполнения exec-once и exec-multiple. Третьим важным элементом стало корректное формирование пользовательского стека и передача аргументов. В start_process() команда запуска разбивается на отдельные слова, строки копируются в стек, формируется массив указателей argv[], устанавливается argc и обеспечивается правильное выравнивание стека. Это обеспечивает корректную работу программы child-args, которая проверяет
exec-bad-ptr	Тест проверяет устойчивость системного вызова exec() к передаче некорректного указателя из пользовательской программы. В тесте вызывается exec((char*) 0x20101234), то есть в качестве аргумента передаётся заведомо недействительный адрес, не принадлежащий пользовательскому адресному пространству. Согласно требованиям Pintos, любая попытка ядра разыменовать недопустимый адрес, переданный из пользовательского процесса, должна привести к немедленному завершению процесса с кодом -1.	

		правильность структуры аргументов, и позволяет пройти тест exec-arg. Дополнительно в обработчике системных вызовов была реализована проверка корректности указателей, передаваемых пользователем. Перед разыменованием аргумента cmd_line вызывается проверка принадлежности адреса пользовательскому пространству и его доступности. При обнаружении недопустимого указателя процесс немедленно завершается с кодом -1, что соответствует требованиям теста exec-bad-ptr и предотвращает повреждение ядра.
wait-simple	Тест проверяет, что wait(exec("child-simple"))	Реализована структура child_status и механизм

	корректно дожидается завершения дочернего процесса и возвращает его код выхода. Родитель запускает программу child-simple, затем сразу вызывает wait() и ожидает получить статус, переданный ребёнком через exit().	синхронизации: дочерний процесс при завершении сохраняет exit_status и вызывает sema_up(), а родитель в wait() блокируется на sema_down() до завершения ребёнка. После пробуждения wait() возвращает сохранённый статус. Благодаря этому wait-simple получает правильный результат.
wait-twice	Тест проверяет, что процесс может ожидать завершение ребёнка только один раз. Первый вызов wait(child) должен корректно дождаться выполнения child-simple и вернуть его код выхода. Второй вызов wait(child) должен сразу вернуть -1, так как ребёнок уже был ожидаем и больше не доступен для повторного wait().	В структуре child_status используется флаг waited, который отмечает, что данный ребёнок уже был ожидаем родителем. При первом вызове wait() родитель блокируется на семафоре, получает код выхода и помечает waited = true. При повторном вызове wait(child) проверка обнаруживает, что ребёнок уже был

		ожидаем, и функция немедленно возвращает -1. Этим обеспечивается корректное прохождение теста *wait-twice*.
wait-killed	Тест проверяет, что wait() корректно обрабатывает ситуацию, когда дочерний процесс был аварийно завершён ядром из-за недопустимого поведения. В тесте запускается программа child-bad, которая выполняет запрещённые действия и завершается не через exit(), а принудительно. В таком случае wait(pid) должен вернуть -1.	При завершении дочернего процесса, независимо от причины, process_exit() записывает его код завершения в child_status. Если процесс был уничтожен ядром, ядро автоматически устанавливает exit_status = -1. Родитель, вызвав wait(pid), блокируется на семафоре до конца выполнения ребенка и затем получает сохранённый статус. Благодаря этому wait() возвращает -1 для child-bad, что соответствует требованиям теста *wait-killed*.
wait-bad-pid	Тест проверяет обработку	В реализации wait(pid)

	вызова wait() с PID, который не является дочерним процессом вызывающего потока. Пользовательская программа вызывает wait(0x0c020301), то есть заведомо некорректный PID. Ожидаемое поведение: системный вызов должен немедленно вернуть -1 или завершить текущий процесс с кодом -1, если проверка адреса или PID невозможна.	выполняется поиск структуры child_status в списке потомков текущего процесса. Если запись не найдена, то это означает, что указанный PID не является дочерним. В этом случае функция сразу возвращает -1, не блокируясь и не вызывая ошибок ядра. Такой подход предотвращает некорректное ожидание произвольных процессов и обеспечивает успешное прохождение теста *wait-bad-pid*.
create-empty	Тест проверяет поведение системного вызова create() при попытке создать файл с пустым именем (""). По спецификации файловой системы Pintos, файл с пустым именем создать нельзя, поэтому системный вызов должен вернуть false, не создавая никакого объекта.	В обработчике SYS_CREATE выполняется проверка строки имени файла с помощью validate_user_string(). Пустая строка корректна как указатель, но её длина равна нулю, и функция

		filesys_create("", size) возвращает false. Значение сохраняется в eax, и процесс продолжает работу.
create-long	Тест проверяет реакцию системного вызова create() на слишком длинное имя файла. В тесте формируется строка размером 512 байт, полностью заполненная символами 'x'. Такое имя превышает максимально допустимую длину для файловой системы Pintos, поэтому create() должен вернуть false.	При обработке SYS_CREATE строка имени сначала проходит проверку валидности через validate_user_string(), после чего передаётся в файловую систему. Функция filesys_create() сравнивает длину имени с максимально допустимой и возвращает false, если имя слишком длинное. Значение false записывается в eax, что соответствует ожидаемому результату теста **create-long**.
create-normal	Тест проверяет стандартное поведение create() при создании обычного пустого файла с валидным именем ("quux.dat"). Ожидается, что файл успешно создастся, а	Обработчик SYS_CREATE проверяет корректность указателей и строки имени, затем вызывает filesys_create(). Так как

	вызов create() вернёт true.	имя корректно и длина не превышена, файловая система создаёт новый файл и возвращает true. Это значение попадает в eax, и тест **create-normal** проходит успешно.
create-exists	Тест проверяет, что create() корректно обрабатывает ситуацию, когда файл с указанным именем уже существует. Сначала создаются файлы "quux.dat", "warble.dat" и "baffle.dat", после чего выполняются повторные вызовы create() с теми же именами. Если файл уже существует, системный вызов обязан вернуть false.	После проверки указателей и строки имени обработчик SYS_CREATE вызывает filesys_create(). Файловая система Pintos возвращает false, если файл с таким именем уже существует. Обработчик передаёт это значение в eax, что приводит к ожидаемому результату: первый вызов create(name) возвращает true, а попытка повторного создания — false. Благодаря этому тест **create-exists** успешно проходит.
create-null	Тест проверяет реакцию системного вызова create() на	Обработчик SYS_CREATE вызывает

	передачу NULL вместо имени файла. Передача нулевого указателя считается ошибкой пользователя, и по правилам Pintos процесс должен быть немедленно завершён с кодом -1.	validate_user_pointer() и validate_user_string() для имени файла. Переданный NULL не является корректным пользовательским адресом, поэтому проверка приводит к аварийному завершению процесса через exit(-1). Это поведение соответствует требованиям теста **create-null**.
create-bad-ptr	Тест передаёт в create() заведомо неверный указатель (0x20101234), который не принадлежит пользовательской памяти. По правилам Pintos любое разыменование некорректного указателя в системном вызове должно привести к немедленному завершению процесса с кодом -1.	В обработчике SYS_CREATE выполняется проверка адреса имени файла через validate_user_pointer() и validate_user_string(). Поскольку указанный адрес недействителен, проверка обнаруживает ошибку и вызывает аварийное завершение процесса через exit(-1). Благодаря этому тест **create-bad-ptr**

		завершается корректно.
create-bound	Тест проверяет, что системный вызов create() корректно обрабатывает строку имени файла, которая находится на границе двух страниц пользовательской памяти. Функция copy_string_across_boundary() специально формирует строку так, что она начинается в конце одной страницы и заканчивается в следующей. Это полностью валидная ситуация, и create() должен успешно создать файл.	В обработчике SYS_CREATE строка имени проверяется с помощью validate_user_string(), которая безопасно проходит строку байт за байтом, каждый раз проверяя, что адрес принадлежит пользовательскому пространству. Такой пошаговый контроль позволяет корректно прочитать строку, даже если она пересекает границу страниц. После успешной проверки имя передаётся в filesys_create(), которая создаёт файл и возвращает true. Благодаря этому тест **create-bound** выполняется успешно.
open-missing	Тест проверяет корректную обработку системного вызова open() при попытке открыть несуществующий файл.	В обработчике SYS_OPEN выполняется проверка указателя и строки

	Пользовательская программа вызывает open("no-such-file"), и ожидается, что системный вызов вернёт **-1**, поскольку файла не существует.	имени через validate_user_pointer() и validate_user_string(). Далее вызывается filesys_open(), которая возвращает NULL, если файла не существует. В этом случае обработчик записывает в eax значение **-1**, не создаёт структуру file_descriptor и не добавляет её в список описателей. Благодаря этому тест **open-missing** успешно проходит.
open-normal	Тест проверяет, что системный вызов open() корректно открывает существующий файл. При вызове open("sample.txt") должен быть возвращён валидный файловый дескриптор, номер которого ≥ 2 (дескрипторы 0 и 1 заняты stdin/stdout). Если дескриптор меньше 2 или равно -1, тест считается проваленным.	В обработчике SYS_OPEN производится проверка указателя и строки имени, затем вызывается filesys_open(). Если файл существует, файловая система возвращает объект struct file *. Обработчик создаёт пользовательский дескриптор, сохраняет

		его в таблице открытых файлов текущего процесса и возвращает его индекс в eax. Для существующего файла дескриптор всегда ≥ 2, поэтому тест **open-normal** проходит успешно.
open-twice	Тест проверяет, что один и тот же файл можно открыть несколько раз, и каждый вызов open("sample.txt") возвращает новый, уникальный файловый дескриптор. Два возвращённых дескриптора не должны совпадать, иначе это означает неправильное управление открытыми файлами.	При каждом вызове SYS_OPEN происходит вызов filesys_open(), который создаёт новое файловое состояние (struct file) независимо от предыдущих открытий. Обработчик системного вызова добавляет каждый объект file в таблицу открытых файлов процесса и выдаёт новый номер дескриптора. Поскольку каждый вызов создаёт отдельный дескриптор, значения h1 и h2 различаются, и тест **open-twice** проходит успешно.

open-boundary	Тест проверяет, что open() корректно работает со строкой имени файла, расположенной на границе двух страниц пользовательской памяти. Функция copy_string_across_boundary() формирует строку "sample.txt" так, что она начинается в конце одной страницы и продолжается в следующей. Такая ситуация допустима, и вызов open() должен успешно вернуть валидный файловый дескриптор (>1).	В обработчике SYS_OPEN имя файла проверяется функцией validate_user_string(), которая проходит строку побайтно и гарантирует, что каждый адрес относится к пользовательскому пространству. Благодаря этому имя корректно читается, даже если переходит через границу страниц. После проверки вызывается filesys_open(), которая успешно открывает файл и возвращает объект struct file, а обработчик выдаёт новый файловый дескриптор. Поэтому тест **open-boundary** проходит успешно.
open-empty	Тест проверяет поведение open() при попытке открыть файл с пустым именем (""). Такое имя недопустимо, поэтому системный вызов обязан вернуть -1, не создавая	В обработчике SYS_OPEN строка имени проходит проверку через validate_user_string(), которая принимает

	и не открывая никакой файл.	пустую строку как корректный указатель, но файловая система filesys_open() не может открыть файл с именем длиной 0 и возвращает NULL. Обработчик интерпретирует это как ошибку и записывает в eax значение -1. Благодаря этому тест **open-empty** выполняется корректно.
open-null	Тест вызывает системный вызов open(NULL), то есть передает ядру нулевой указатель вместо имени файла. Требование Pintos: любой системный вызов, получивший недопустимый адрес из пространства пользователя, обязан немедленно завершить процесс с кодом -1. Поэтому корректное поведение — аварийное завершение вызывающего процесса.	Системный вызов open должен проверять валидность указателя на строку через validate_user_pointer() и validate_user_string(). Переданный NULL не является корректным пользовательским адресом, поэтому проверка должна вызвать syscall_exit(-1) или аналогичный механизм завершения процесса. В результате процесс завершается с кодом -1, и тест

		засчитывается как успешный.
open-bad-ptr	Тест вызывает open((char*)0x20101234), то есть передаёт в системный вызов заведомо невалидный пользовательский указатель, указывающий за пределы адресного пространства процесса. Согласно требованиям Pintos, любое обращение системного вызова к недопустимому пользовательскому адресу должно немедленно завершить процесс с кодом -1. Если ядро попытается разыменовать этот адрес или не завершит процесс, тест должен упасть.	В обработчике SYS_OPEN необходимо проверить указатель на имя файла функциями validate_user_pointer() и validate_user_string(). Поскольку адрес 0x20101234 не принадлежит пространству пользователя, проверка должна вызвать аварийное завершение процесса через syscall_exit(-1). Процесс завершается с кодом -1, и тест проходит.
close-normal	Тест открывает файл sample.txt, убеждается, что системный вызов open() вернул корректный файловый дескриптор (строго больше 1), выводит сообщение и затем вызывает close(handle). Проверяется, что закрытие открытого файла проходит корректно: дескриптор освобождается, запись об	В обработчике SYS_CLOSE необходимо убедиться, что дескриптор валиден и относится к текущему процессу, затем закрыть соответствующий файл через file_close() и удалить запись о нём из таблицы открытых файлов. После

	открытом файле удаляется из таблицы файлов процесса, и ресурс действительно отпускается. Ошибок возникнуть не должно.	корректной реализации закрытие обычного файла выполняется успешно, что и подтверждает прохождение теста close-normal.
close-twice	Тест открывает файл sample.txt, корректно закрывает его, а затем вызывает close() по тому же файловому дескриптору ещё раз. Ожидается, что второе закрытие **не приведёт к краху**: либо оно тихо игнорируется, либо процесс завершается с кодом -1 (в зависимости от реализации проверки валидности дескриптора). Главное — ядро не должно работать с уже освобождённой структурой файла.	В обработчике SYS_CLOSE необходимо проверять, существует ли запись для данного дескриптора в таблице открытых файлов процесса. Если дескриптор уже был закрыт, следует безопасно вернуть управление без обращения к file_close(). Такая защита предотвращает двойное освобождение и гарантирует успешное прохождение теста close-twice.
close-stdin	Тест вызывает close(0), пытаясь закрыть стандартный поток ввода (stdin). По требованиям Pintos, такое	В обработчике SYS_CLOSE необходимо явно проверять, что

	закрытие **не должно приводить к некорректному поведению**: ядро либо тихо игнорирует операцию, либо завершает процесс с кодом -1. Главное — система не должна обращаться к несуществующему объекту файла и не должна падать.	файловый дескриптор 0 не соответствует обычному открытому файлу. Если stdin не имеет связанного объекта struct file, вызов следует безопасно игнорировать: не выполнять file_close() и не пробовать освободить запись. Такая проверка предотвращает обращение к NULL и гарантирует прохождение теста close-stdin.
close-stdout	Тест вызывает close(1), пытаясь закрыть стандартный поток вывода (stdout). Аналогично stdin, этот файловый дескриптор не связан с реальным объектом файловой системы. По требованиям Pintos операция должна быть безопасной: ядро либо тихо игнорирует её, либо завершает процесс с кодом -1. Главное — не допустить обращения к	В обработчике SYS_CLOSE выполняется проверка, является ли закрываемый дескриптор 1 стандартным выводом. Если это stdout, ядро **не должно вызывать file_close()** и не должно удалять запись о файле, поскольку фактического файла за этим

	неинициализированной структуре файла и не вызвать ошибку доступа.	дескриптором нет. Просто пропускаем операцию, не делая никаких действий. Это гарантирует корректное прохождение теста close-stdout.
close-bad-fd	Тест вызывает close(0x20101234) — то есть передает в системный вызов **заведомо несуществующий файловый дескриптор**. По правилам Pintos такая операция не должна приводить к обращению к памяти ядра через неверный индекс и не должна вызывать ошибку. Ядро обязано либо **безопасно проигнорировать** вызов, либо завершить процесс с кодом -1. Главное — не производить доступ к структурам, которых не существует.	В обработчике SYS_CLOSE перед любыми действиями проверяется, что fd находится в допустимых пределах и что в таблице файлов процесса за ним действительно закреплён открытый файл. Если fd некорректен, функция просто завершает выполнение без вызова file_close(). Такая «тихая неоперация» полностью соответствует требованиям теста close-bad-fd.
read-normal	Тест *read-normal* проверяет базовую корректность работы системного вызова read().	Для прохождения теста необходимо корректно реализовать обработку

	Пользовательская программа открывает существующий файл *sample.txt* и пытается прочитать его содержимое в буфер. Затем тест сравнивает полученные данные с эталонной строкой из sample.inc. Если чтение прошло правильно, количество возвращённых байт и содержимое буфера полностью совпадают с ожидаемыми.	системного вызова SYS_READ. Ядро должно проверить валидность пользовательского буфера и файла, определить тип источника данных (файл или стандартный ввод) и выполнить чтение через file_read(), возвращая точное число реально прочитанных байт. Драйвер файловой системы обязан корректно вести текущую позицию в файле. При корректной обработке всех указателей и возвращаемых значений тест *read-normal* успешно проходит.
read-zero	Тест *read-zero* проверяет, корректно ли реализовано чтение нулевого количества байт. Программа открывает файл *sample.txt*, устанавливает в буфер известное значение и вызывает read() с параметром	Для прохождения теста реализация системного вызова SYS_READ должна обрабатывать случай size == 0 как особый: функция обязана сразу вернуть 0

	размера равным нулю. Ожидается, что системный вызов немедленно вернёт 0, не изменяя содержимое буфера и не обращаясь к файлу фактически.	и не выполнять никаких операций чтения. Буфер и позиция файла должны остаться неизменными. Такая проверка предотвращает лишние обращения к файловой системе и соответствует стандартному поведению POSIX. Корректное выполнение этого условия гарантирует прохождение теста *read-zero*.
read-bad-ptr	Тест *read-bad-ptr* передаёт в системный вызов read() явно недопустимый указатель (0xC0100000). Такое значение не принадлежит пользовательскому адресному пространству, поэтому корректная ОС обязана немедленно завершить процесс с кодом -1, не пытаясь выполнить чтение. Цель теста — убедиться, что ядро проверяет валидность пользовательских буферов	Чтобы пройти тест, реализация SYS_READ должна вызывать проверку указателя (validate_user_pointer() или validate_user_buffer()) перед тем, как пытаться читать данные. Если указатель выходит за пределы пользовательской памяти или недоступен для записи, ядро должно

	перед тем, как обращаться к ним.	аварийно завершить процесс через exit(-1). При корректной проверке выполнение не дойдёт до чтения и тест будет успешно пройден.
read-boundary	Тест *read-boundary* проверяет, сможет ли системный вызов read() корректно заполнить пользовательский буфер, который расположен на границе двух страниц виртуальной памяти. Буфер начинается почти в конце одной страницы и продолжается на следующую, но такое расположение является допустимым, поэтому чтение должно завершиться успешно. После выполнения операции тест сравнивает содержимое буфера с эталонной строкой, убеждаясь, что данные были прочитаны полностью и без искажений.	Для прохождения теста ядро должно проверять корректность всего диапазона буфера, а не только его начального адреса. Каждая страница, входящая в область записи, обязана быть доступной пользователю; только после этого можно выполнять чтение. Правильная реализация системного вызова читает данные в буфер, даже если он пересекает границу страниц, и обеспечивает точное копирование текста. Когда проверка адресов постранична и обработка чтения корректна, тест завершается успешно.

read-stdout	Тест *read-stdout* пытается выполнить чтение из файлового дескриптора 1, который соответствует стандартному выводу (stdout). По договорённости Pintos, stdout предназначен только для записи, поэтому операция чтения из него является некорректной. Такое обращение либо должно вернуть ошибку, либо немедленно завершить процесс с кодом -1. Тест проверяет, что реализация системного вызова read() не допускает чтение из stdout как из обычного файла.	Для прохождения теста обработчик read() обязан распознавать, что файловый дескриптор 1 относится к каналу вывода и не является читаемым источником. Правильная реализация либо возвращает 0 или -1 без изменения буфера, либо завершает процесс через exit(-1) в соответствии с политикой проверки указателей и валидности дескрипторов. При наличии корректной проверки допустимых каналов ввода тест стабильно проходит.
read-bad-fd	Тест *read-bad-fd* вызывает системный вызов read() с различными заведомо некорректными файловыми дескрипторами: слишком большими числами, отрицательными значениями, случайными адресами и др. Ни один из них не соответствует	Чтобы тест проходил, реализация SYS_READ должна проверять валидность файлового дескриптора перед чтением. Конкретно: дескриптор должен быть неотрицательным, меньше размера

	открытому файлу или стандартным потокам ввода/вывода. Задача теста — убедиться, что ядро Pintos корректно обрабатывает подобные случаи и не позволяет читать из несуществующих или недействительных дескрипторов. Поведение считается правильным, если чтение завершается ошибкой или процесс завершается с кодом -1.	таблицы открытых файлов текущего процесса и должен соответствовать реально открытому файлу. Если дескриптор невалиден, функция обязана вернуть -1 или завершить процесс через exit(-1) согласно логике проверки пользовательских параметров. При такой проверке любые обращения к мусорным или непринадлежащим процессу дескрипторам не приводят к чтению, и тест успешно проходит.
write-normal	Тест *write-normal* проверяет, способна ли система корректно выполнять обычную запись в файл. Пользовательская программа создаёт файл, открывает его и вызывает write() с буфером, содержащим заранее известный текст. Ожидается, что системный вызов запишет весь переданный объём данных и	Для прохождения теста обработка системного вызова write() должна включать проверку валидности указателей и корректность файлового дескриптора, определение реального открытого файла через таблицу дескрипторов, а затем выполнение

	вернёт количество записанных байт, полностью совпадающее с размером буфера.	записи с помощью file_write(). Возвращаемое значение должно отражать фактическое число записанных байт. При такой реализации обычная запись выполняется без ошибок, что обеспечивает успешное выполнение теста *write-normal*.
write-zero	Тест *write-zero* проверяет поведение системного вызова write() при попытке записать нулевое количество байт. Программа открывает файл, подготавливает буфер и вызывает write() с аргументом размера, равным нулю. Ожидается, что системный вызов немедленно вернёт 0, не изменяя буфер и не выполняя фактической записи в файл.	Для успешного прохождения теста обработка write() должна распознавать случай нулевого размера как корректную, но пустую операцию. В этом случае запись не производится, файловая позиция не меняется, а результатом вызова является возврат значения 0. Такое поведение соответствует общепринятой

		семантике системного вызова и обеспечивает выполнение теста *write-zero* без ошибок.
write-bad-ptr	Тест *write-bad-ptr* передаёт в системный вызов write() заведомо недопустимый указатель на пользовательский буфер — адрес, находящийся вне области пользовательской памяти. Ожидаемое поведение состоит в немедленном завершении процесса с кодом -1, поскольку ядро не имеет права dereference'ить такой указатель и продолжать выполнение.	Для успешного прохождения теста реализация системного вызова должна выполнять обязательную проверку валидности пользовательского буфера перед попыткой записи. Если указатель указывает на недоступный или несуществующий участок пользовательской памяти, обработка должна прервать выполнение программы посредством exit(-1) без обращения к файловой системе. Такое поведение гарантирует, что некорректный буфер не будет использован, и тест *write-bad-ptr* завершается правильно.

write-boundary	Тест *write-boundary* проверяет работу системного вызова write() при записи данных, расположенных на границе двух пользовательских страниц. Буфер формируется так, что его начало находится в конце одной страницы, а окончание — уже в следующей. Несмотря на такое разбиение, операция записи должна завершиться успешно, а количество записанных байт обязано совпадать с размером переданного буфера.	Для успешного выполнения теста реализация write() должна проверять корректность всего диапазона пользовательского буфера, включая каждую страницу, по которой проходит запись. Если все адреса доступны для чтения в пользовательском пространстве, то данные должны быть переданы в файловую систему без учёта границ страниц. Корректная постраничная проверка буфера и правильная передача данных обеспечивают успешное прохождение теста *write-boundary*.
write-stdin	Тест *write-stdin* пытается выполнить запись в файловый дескриптор 0, который соответствует стандартному потоку ввода (stdin). По правилам Pintos запись в stdin	Для корректной обработки такого случая реализация должна проверять номер файлового дескриптора. Если он равен 0,

	недопустима: поток предназначен только для чтения. Поэтому вызов write(0, …) должен либо просто вернуть ошибку, либо немедленно завершить процесс с кодом -1.	операция записи не выполняется: либо возвращается ошибка, либо процесс завершается. Такая проверка предотвращает некорректную работу с потоками ввода-вывода и обеспечивает успешное прохождение теста *write-stdin*.
write-bad-fd	Тест *write-bad-fd* вызывает write() с набором некорректных файловых дескрипторов: слишком большими числами, отрицательными значениями, а также заведомо “мусорным” адресом 0x01012342, который вообще не является FD. В каждом таком вызове система должна либо спокойно вернуть ошибку (обычно 0 или -1), либо завершить процесс с кодом -1, если ошибка критическая. Главное требование — никакого чтения и записи в невалидные структуры, никакого обращения к kernel-памяти по	Правильная обработка достигается через проверку диапазона файловых дескрипторов и существования элемента в таблице открытых файлов процесса. Если номер FD меньше 2 или превышает максимально допустимый, либо соответствующий элемент таблицы равен NULL, операция записи не выполняется, и система возвращает ошибочный результат. Благодаря этому обращение к

	неверным индексам.	несуществующим дескрипторам не приводит к сбою, и тест *write-bad-fd* проходит успешно.

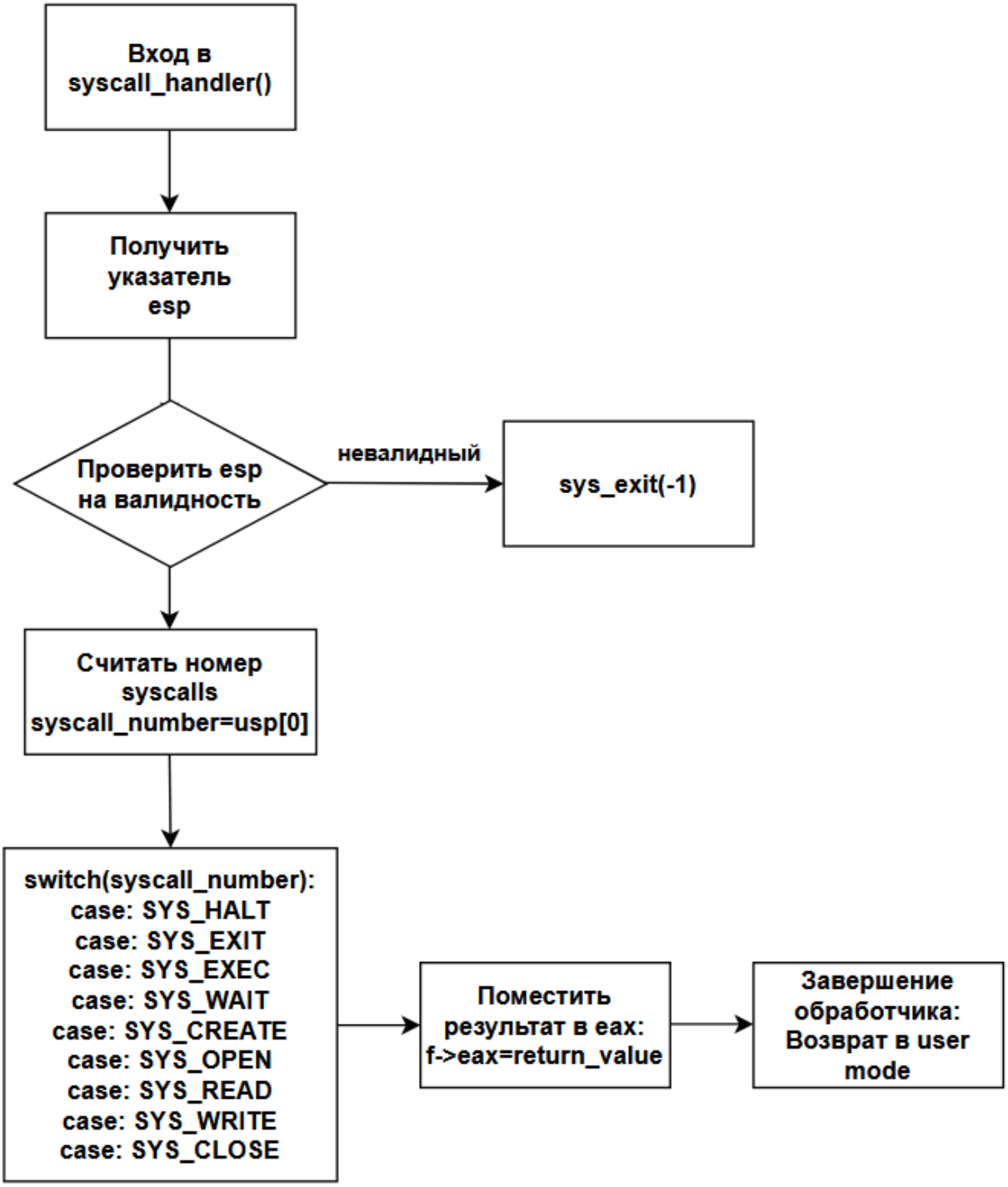


Рис. 3 «Блок-схема реализованного механизма обработки системных вызовов»

Индивидуальное дополнительное задание

Было выполнено дополнительное индивидуальное задание. Была реализован редактор (editor.c). Программа позволяет: создавать файлы для хранения строк, считывать, вставлять строки файла, удалять строки файла, записывать в строки файла, получать строку файла. Примеры работы программы:

```
Executing 'shell':
Shell starting...
--editor help
editor help
editor create [filename] [max_lines] [max_line_length]
editor info [filename]
editor lines [filename]
editor read [filename] [line_number]
editor del [filename] [line_number]
editor ins [filename] [line_number] [text]
editor: exit(0)
"editor help": exit code 0
```

Рис. 4 «editor help»

```
--editor create korpusovIBAS 100 100
editor: exit(0)
"editor create korpusovIBAS 100 100": exit code 0
```

Рис. 5 «editor create»

```
"editor ins korpusovIBAS 1 secret": exit code 0
--editor read korpusovIBAS 1
secret
editor: exit(0)
"editor read korpusovIBAS 1": exit code 0
```

Рис. 6 «editor ins и editor read»

```
--editor info korpusovIBAS
max lines: 100
max line length: 100
current lines: 1
editor: exit(0)
"editor info korpusovIBAS": exit code 0
```

Рис. 7 «editor info»

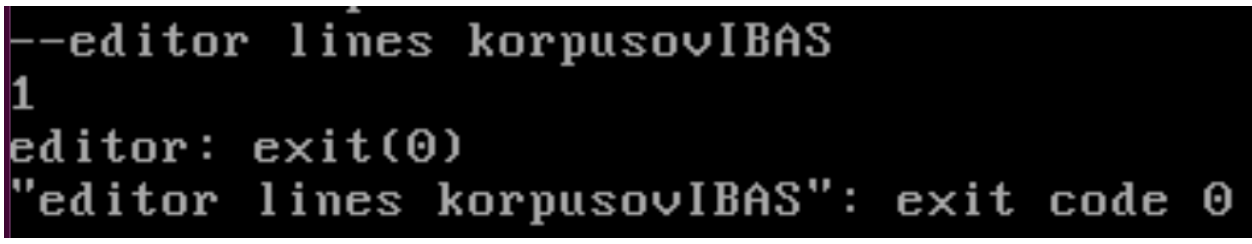


Рис. 5 «editor lines»

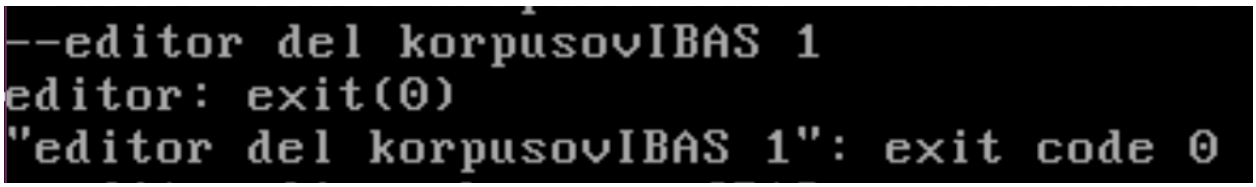


Рис. 5 «editor del»

ВЫВОД

В ходе работы был реализован полный механизм обработки системных вызовов пользовательских программ. Обработчик системных вызовов получает номер операции и аргументы из пользовательского стека, выполняет проверку корректности указателей и вызывает соответствующие функции ядра. Были добавлены реализации системных вызовов для работы с процессами (halt, exit, exec, wait) и файлами (create, open, read, write, close), включая обработку ошибок, проверку границ памяти и защиту от некорректных дескрипторов. Реализация была проверена набором стандартных тестов Pintos, охватывающих корректное выполнение, обработку ошибок и пограничные случаи. Все тесты были успешно пройдены, что подтверждает корректность механизма системных вызовов и его соответствие требованиям.

Листинг А «syscall.c»

```
#include "userprog/syscall.h"
#include <debug.h>
#include <stdio.h>
#include <stdint.h>
#include <syscall-nr.h>
#include "threads/interrupt.h"
#include "threads/thread.h"
#include "threads/vaddr.h"
#include "userprog/pagedir.h"
#include "userprog/process.h"
#include "devices/shutdown.h"


static void syscall_handler (struct intr_frame *);
static void validate_user_pointer(const void *uaddr);
static void validate_user_string(const char *uaddr);
static void sys_exit(int statis) NO_RETURN;
static struct file_descriptor *lookup_fd(int fd);
static void close_descriptor (struct file_descriptor *fd);

/* Таблица открытых файлов процесса */
struct file_descriptor
{
     int fd;                // Файловый дескриптор
     struct file *file;
     struct list_elem elem;
};

static struct lock filesys_lock;

void
syscall_init (void)
{
  intr_register_int    (0x30,    3,    INTR_ON,    syscall_handler,
"syscall");
  lock_init(&filesys_lock);
}

static void
syscall_handler (struct intr_frame *f)
{
  uint32_t *usp = (uint32_t *) f->esp;

  validate_user_pointer(usp);
  int syscall_number = usp[0];

  switch(syscall_number)
  {
  case SYS_HALT:  // Завершение работы ОС и выключение VM
     shutdown_power_off();
     break;
```

```
  case SYS_EXIT:
    validate_user_pointer(usp+1);  //  проверяем  первый  аргумент
системного вызова
    sys_exit((int)usp[1]);
    break;
  case SYS_OPEN:
     validate_user_pointer(usp+1);   //проверяем  первый  аргумент
системн
     validate_user_string((const  char  *)  usp[1]);   //  Проверяем
имя файла
     struct file *file;
     struct file_descriptor *fd_entry;

     lock_acquire(&filesys_lock);
     file=filesys_open((const char *) usp[1]);
     lock_release(&filesys_lock);

     if (file == NULL)
     {
          f->eax = -1;
          break;
     }

     fd_entry = malloc(sizeof *fd_entry);
     if (fd_entry == NULL)
     {
          lock_acquire(&filesys_lock);
          file_close(file);
          lock_release(&filesys_lock);
          f->eax=-1;
          break;
     }

     fd_entry->fd = thread_current()->next_fd++;
     fd_entry->file=file;
     list_push_back(&thread_current()-
>file_descriptors,&fd_entry->elem);
     f->eax = fd_entry->fd;
     break;

  case SYS_EXEC:
     validate_user_pointer(usp+1);
     validate_user_string((const char *) usp[1]);
     f->eax = process_execute((const char *) usp[1]);
     break;

  case SYS_WAIT:
     validate_user_pointer(usp+1);
     f->eax = process_wait((tid_t)usp[1]);
     break;

  case SYS_READ:
```

```
    validate_user_pointer(usp+1);
    validate_user_pointer(usp+2);
    validate_user_pointer(usp+3);
    {
          int fd = (int) usp[1];
          void *buffer = (void *) usp[2];
          unsigned size = (unsigned) usp[3];


          if(size>0)
          {
               validate_user_pointer((const uint8_t *) buffer);
               validate_user_pointer((const uint8_t *) buffer +
size - 1);
          }
          if (fd==0)
          {
               unsigned i;
               uint8_t *buf = buffer;
               for(i=0;i<size;i++)
                    buf[i]=input_getc();
               f->eax=(int)size;
          }
          else if(fd==1)
               f->eax = -1;
          else
          {
               struct file_descriptor *file_desc = lookup_fd(fd);
               if(file_desc == NULL)
                    f->eax=-1;
               else
               {
                    lock_acquire(&filesys_lock);
                    f->eax=file_read(file_desc-
>file,buffer,size);
                    lock_release(&filesys_lock);
               }
          }

     }
     break;



  case SYS_CREATE:
     validate_user_pointer(usp+1);
     validate_user_pointer(usp+2);
     validate_user_string((const char*) usp[1]);
     lock_acquire(&filesys_lock);
     f->eax = filesys_create((const char*) usp[1], (unsigned)
usp[2]);
     lock_release(&filesys_lock);
     break;
```

```
  case SYS_REMOVE:
    validate_user_pointer(usp+1);
    validate_user_string((const char *) usp[1]);
    lock_acquire(&filesys_lock);
    f->eax = filesys_remove((const char*) usp[1]);
    lock_release(&filesys_lock);
    break;

  case SYS_FILESIZE:
    validate_user_pointer(usp+1);
    {
          struct file_descriptor *fd=lookup_fd((int) usp[1]);
          if (fd == NULL)
               f->eax=-1;
          else
          {
               lock_acquire(&filesys_lock);
               f->eax=file_length(fd->file);
               lock_release(&filesys_lock);
          }
     }
     break;

  case SYS_WRITE: //write(fd(stdin-0,stdout-1), buffer, size)
    validate_user_pointer(usp+1);
    validate_user_pointer(usp+2);
    validate_user_pointer(usp+3);
    {
      int fd = (int) usp[1];
      const void *buffer = (const void *) usp[2];
      unsigned size = (unsigned) usp[3];

      if (size>0)
      {
        validate_user_pointer((const  uint8_t  *)  buffer);  //
проверяем начало буфера
        validate_user_pointer((const uint8_t *) buffer +size -
1); // проверяем конец буфера
      }

      if (fd==1)
      {
        putbuf(buffer,size);
        f->eax = (int) size;
      }
      else if(fd==0)
        f->eax = -1;
      else
      {
          struct file_descriptor *file_desc=lookup_fd(fd);
          if (file_desc == NULL)
               f->eax=-1;
```

```
          else
          {
               lock_acquire(&filesys_lock);
               f->eax=file_write(file_desc->file, buffer, size);
               lock_release(&filesys_lock);
          }
       }
    }
    break;
  case SYS_CLOSE:
    validate_user_pointer(usp+1);
    {
          int fd = (int) usp[1];
          if(fd==1)
               sys_exit(-1);

          struct file_descriptor *file_desc = lookup_fd(fd);
          if (file_desc!=NULL)
               close_descriptor(file_desc);
     }
     break;

  default:
    sys_exit(-1);
    break;
  }
}

static void validate_user_pointer(const void *uaddr)
{
  if (uaddr == NULL || !is_user_vaddr(uaddr)
      ||       pagedir_get_page       (thread_current()->pagedir,
uaddr)==NULL)
  {
    sys_exit(-1);
  }
}

static void validate_user_string(const char *uaddr)
{
     while(true)
     {
          validate_user_pointer(uaddr);
          if(*uaddr=='\0')
               break;

          void* next_page=pg_round_up(uaddr+1);
          while((void *) uaddr<next_page)
          {
               validate_user_pointer(uaddr);
               if(*uaddr=='\0')
                    return;
               uaddr++;
```

```
          }

     }
}


static void sys_exit(int status)
{
  thread_current()->exit_status = status;
  thread_exit();
}

void syscall_close_all_files (void)
{
     struct thread *cur = thread_current();

     while(!list_empty (&cur->file_descriptors))
     {
          struct          file_descriptor          *fd          =
list_entry(list_begin(&cur->file_descriptors),             struct
file_descriptor, elem);
          close_descriptor(fd);
     }
}

static struct file_descriptor *lookup_fd (int fd)
{
  struct thread *cur = thread_current ();
  struct list_elem *e;

  for (e = list_begin (&cur->file_descriptors);
       e != list_end (&cur->file_descriptors);
       e = list_next (e))
    {
      struct  file_descriptor  *desc  =  list_entry  (e,  struct
file_descriptor, elem);
      if (desc->fd == fd)
        return desc;
    }

  return NULL;
}

static void close_descriptor (struct file_descriptor *fd)
{
  list_remove (&fd->elem);
  lock_acquire (&filesys_lock);
  file_close (fd->file);
  lock_release (&filesys_lock);
  free (fd);
}
```

Листинг Б «editor.c»

```
#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>
#include <syscall.h>

#define MAX_FILE_SIZE (1024 * 1024)

struct editor_header
{
    uint32_t max_lines;
    uint32_t max_line_length;
    uint32_t line_count;
};

struct line
{
    char* data;
    uint32_t length;
};

static char memory_pool[MAX_FILE_SIZE];
static size_t memory_used;

static void
memory_reset(void)
{
    memory_used = 0;
}

static void*
xalloc(size_t size)
{
    size_t aligned = (size + 3) & ~((size_t)3);
    if (memory_used + aligned > sizeof memory_pool)
        return NULL;

    void* ptr = memory_pool + memory_used;
    memory_used += aligned;
    return ptr;
}

static void
print_help(void)
{
    printf("editor help\n");
    printf("editor      create      [filename]      [max_lines]
[max_line_length]\n");
    printf("editor info [filename]\n");
    printf("editor lines [filename]\n");
    printf("editor read [filename] [line_number]\n");
```

```
    printf("editor del [filename] [line_number]\n");
    printf("editor ins [filename] [line_number] [text]\n");
}

static bool
parse_u32(const char* text, uint32_t* value)
{
    uint32_t result = 0;
    const char* p;
    if (text == NULL || *text == '\0')
        return false;

    for (p = text; *p != '\0'; p++)
    {
        if (*p < '0' || *p > '9')
            return false;
        result = result * 10 + (uint32_t)(*p - '0');
    }

    if (result == 0)
        return false;

    *value = result;
    return true;
}

static bool
load_file(const char* filename, struct editor_header* header,
    struct line** lines_out)
{
    int fd = open(filename);
    if (fd < 0)
    {
        printf("%s: open failed\n", filename);
        return false;
    }

    int size = filesize(fd);
    if (size < (int)sizeof * header || size > MAX_FILE_SIZE)
    {
        printf("%s: invalid file size\n", filename);
        close(fd);
        return false;
    }

    char* buffer = xalloc(size);
    if (buffer == NULL)
    {
        printf("out of memory\n");
        close(fd);
        return false;
    }
```

```
    int read_bytes = read(fd, buffer, size);
    close(fd);
    if (read_bytes != size)
    {
        printf("%s: read failed\n", filename);
        return false;
    }

    memcpy(header, buffer, sizeof * header);
    if (header->max_lines == 0 || header->max_line_length == 0
        || header->line_count > header->max_lines)
    {
        printf("%s: invalid header\n", filename);
        return false;
    }

    struct line* lines = xalloc(header->max_lines * sizeof *
lines);
    if (lines == NULL)
    {
        printf("out of memory\n");
        return false;
    }

    memset(lines, 0, header->max_lines * sizeof * lines);

    char* ptr = buffer + sizeof * header;
    char* end = buffer + size;
    uint32_t i;
    for (i = 0; i < header->line_count; i++)
    {
        if (end - ptr < (int)sizeof(uint32_t))
        {
            printf("%s: corrupted file\n", filename);
            return false;
        }

        uint32_t len;
        memcpy(&len, ptr, sizeof len);
        ptr += sizeof len;

        if (len > header->max_line_length || end - ptr <
(int)len)
        {
            printf("%s: corrupted file\n", filename);
            return false;
        }

        lines[i].data = xalloc(len + 1);
        if (lines[i].data == NULL)
        {
            printf("out of memory\n");
            return false;
```

```
        }

        memcpy(lines[i].data, ptr, len);
        lines[i].data[len] = '\0';
        lines[i].length = len;
        ptr += len;
    }

    *lines_out = lines;
    return true;
}

static bool
store_file(const char* filename, const struct editor_header*
header,
    const struct line* lines)
{
    uint32_t i;
    size_t total = sizeof * header;
    for (i = 0; i < header->line_count; i++)
        total += sizeof(uint32_t) + lines[i].length;

    if (total > MAX_FILE_SIZE)
    {
        printf("%s: resulting file too large\n", filename);
        return false;
    }

    remove(filename);
    if (!create(filename, total))
    {
        printf("%s: create failed\n", filename);
        return false;
    }

    int fd = open(filename);
    if (fd < 0)
    {
        printf("%s: open failed\n", filename);
        return false;
    }

    if (write(fd, header, sizeof * header) != (int)sizeof *
header)
    {
        printf("%s: write failed\n", filename);
        close(fd);
        return false;
    }

    for (i = 0; i < header->line_count; i++)
    {
        if (write(fd, &lines[i].length, sizeof lines[i].length)
```

```
            != (int)sizeof lines[i].length)
        {
            printf("%s: write failed\n", filename);
            close(fd);
            return false;
        }
        if (lines[i].length > 0
            &&  write(fd,  lines[i].data,  lines[i].length)  !=
(int)lines[i].length)
        {
            printf("%s: write failed\n", filename);
            close(fd);
            return false;
        }
    }

    close(fd);
    return true;
}

static bool
cmd_create(int argc, char* argv[])
{
    struct editor_header header;
    if (argc != 5 || !parse_u32(argv[3], &header.max_lines)
        || !parse_u32(argv[4], &header.max_line_length))
    {
        printf("usage:  editor  create  [filename]  [max_lines]
[max_line_length]\n");
        return false;
    }

    header.line_count = 0;

    remove(argv[2]);
    if (!create(argv[2], sizeof header))
    {
        printf("%s: create failed\n", argv[2]);
        return false;
    }

    int fd = open(argv[2]);
    if (fd < 0)
    {
        printf("%s: open failed\n", argv[2]);
        return false;
    }

    if (write(fd, &header, sizeof header) != (int)sizeof header)
    {
        printf("%s: write failed\n", argv[2]);
        close(fd);
        return false;
```

```
    }

    close(fd);
    return true;
}

static bool
cmd_info(int argc, char* argv[])
{
    struct editor_header header;
    struct line* lines = NULL;
    if (argc != 3)
        return false;

    memory_reset();
    if (!load_file(argv[2], &header, &lines))
        return false;

    printf("max lines: %u\n", header.max_lines);
    printf("max line length: %u\n", header.max_line_length);
    printf("current lines: %u\n", header.line_count);
    return true;
}

static bool
cmd_lines(int argc, char* argv[])
{
    struct editor_header header;
    struct line* lines = NULL;
    if (argc != 3)
        return false;

    memory_reset();
    if (!load_file(argv[2], &header, &lines))
        return false;

    printf("%u\n", header.line_count);
    return true;
}

static bool
cmd_read(int argc, char* argv[])
{
    struct editor_header header;
    struct line* lines = NULL;
    uint32_t line_no;

    if (argc != 4 || !parse_u32(argv[3], &line_no))
        return false;

    memory_reset();
    if (!load_file(argv[2], &header, &lines))
        return false;
```

```
    if (line_no == 0 || line_no > header.line_count)
    {
        printf("line out of range\n");
        return false;
    }

    printf("%s\n", lines[line_no - 1].data);
    return true;
}

static bool
cmd_del(int argc, char* argv[])
{
    struct editor_header header;
    struct line* lines = NULL;
    uint32_t line_no;

    if (argc != 4 || !parse_u32(argv[3], &line_no))
        return false;

    memory_reset();
    if (!load_file(argv[2], &header, &lines))
        return false;

    if (line_no == 0 || line_no > header.line_count)
    {
        printf("line out of range\n");
        return false;
    }

    uint32_t idx = line_no - 1;
    for (; idx + 1 < header.line_count; idx++)
        lines[idx] = lines[idx + 1];
    header.line_count--;

    return store_file(argv[2], &header, lines);
}

static bool
cmd_ins(int argc, char* argv[])
{
    struct editor_header header;
    struct line* lines = NULL;
    uint32_t line_no;

    if (argc != 5 || !parse_u32(argv[3], &line_no))
        return false;

    memory_reset();
    if (!load_file(argv[2], &header, &lines))
        return false;
```

```
    if (header.line_count >= header.max_lines)
    {
        printf("maximum lines reached\n");
        return false;
    }

    if (line_no == 0 || line_no > header.line_count + 1)
    {
        printf("line out of range\n");
        return false;
    }

    size_t text_len = strlen(argv[4]);
    if (text_len > header.max_line_length)
    {
        printf("line too long\n");
        return false;
    }

    struct line* expanded = xalloc((header.line_count + 1) *
sizeof * expanded);
    if (expanded == NULL)
    {
        printf("out of memory\n");
        return false;
    }

    uint32_t insert_idx = line_no - 1;
    uint32_t i;
    for (i = 0; i < insert_idx; i++)
        expanded[i] = lines[i];

    expanded[insert_idx].data = xalloc(text_len + 1);
    if (expanded[insert_idx].data == NULL)
    {
        printf("out of memory\n");
        return false;
    }
    memcpy(expanded[insert_idx].data, argv[4], text_len + 1);
    expanded[insert_idx].length = (uint32_t)text_len;

    for (; i < header.line_count; i++)
        expanded[i + 1] = lines[i];

    header.line_count++;

    return store_file(argv[2], &header, expanded);
}

int
main(int argc, char* argv[])
{
    if (argc < 2)
```

```
    {
        print_help();
        return EXIT_FAILURE;
    }

    if (!strcmp(argv[1], "help"))
        return print_help(), EXIT_SUCCESS;
    else if (!strcmp(argv[1], "create"))
        return   cmd_create(argc,   argv)   ?   EXIT_SUCCESS   :
EXIT_FAILURE;
    else if (!strcmp(argv[1], "info"))
        return   cmd_info(argc,   argv)   ?   EXIT_SUCCESS   :
EXIT_FAILURE;
    else if (!strcmp(argv[1], "lines"))
        return   cmd_lines(argc,   argv)   ?   EXIT_SUCCESS   :
EXIT_FAILURE;
    else if (!strcmp(argv[1], "read"))
        return   cmd_read(argc,   argv)   ?   EXIT_SUCCESS   :
EXIT_FAILURE;
    else if (!strcmp(argv[1], "del"))
        return   cmd_del(argc,   argv)   ?   EXIT_SUCCESS   :
EXIT_FAILURE;
    else if (!strcmp(argv[1], "ins"))
        return   cmd_ins(argc,   argv)   ?   EXIT_SUCCESS   :
EXIT_FAILURE;

    print_help();
    return EXIT_FAILURE;
}
```